

DESIGN PATTERNS IN C++ – THE COMPOSITE PATTERN

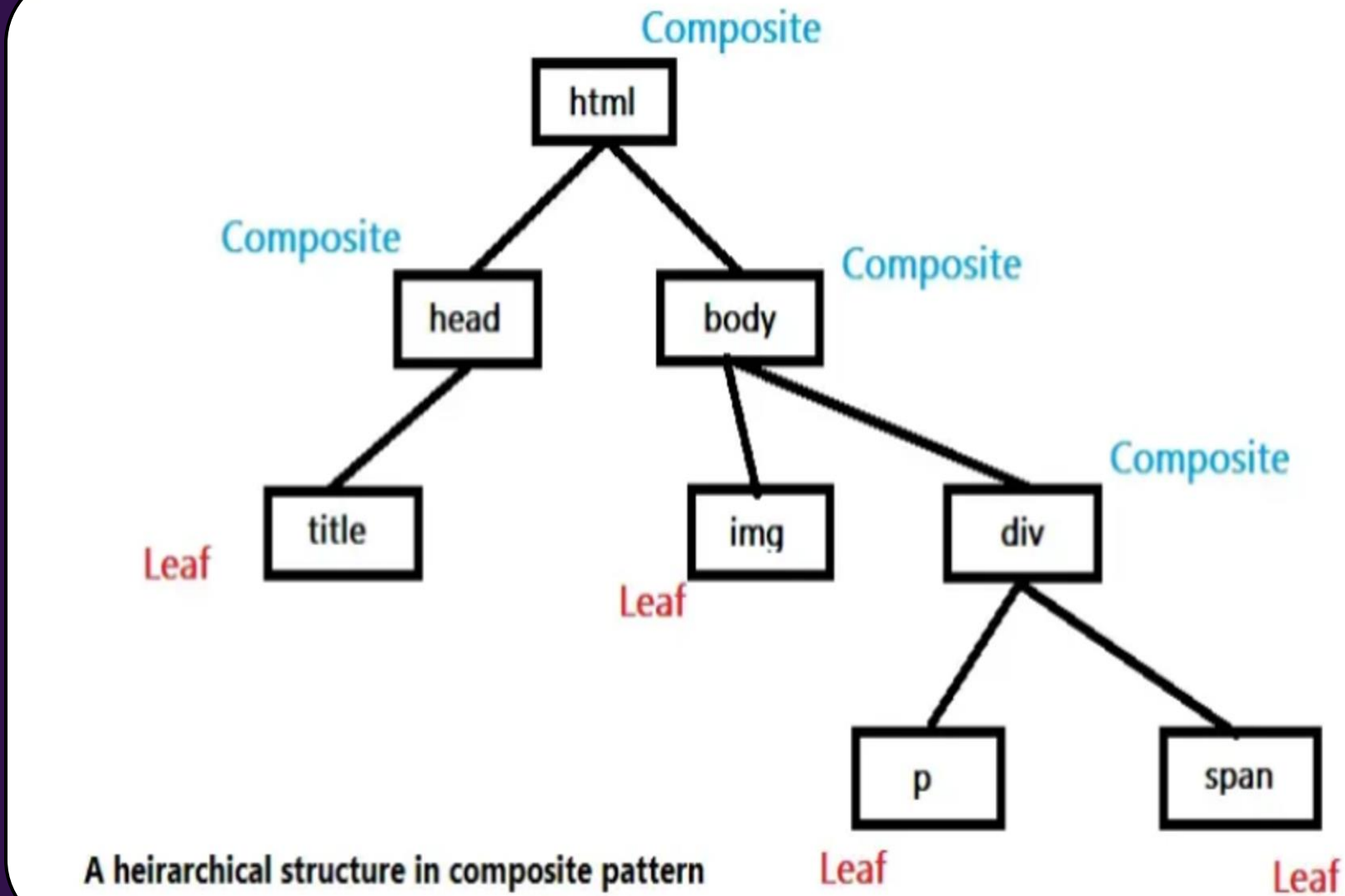
Darryl, Myrthe, Rose, Ryan, Liam, Musawer, Koen

Wat is composition

- Concept in OOP
- Class 'heeft' andere Class

```
class Engine {
public:
    Engine()  { std::cout << "Engine on\n"; }
    ~Engine() { std::cout << "Engine off\n";
}   void start() { std::cout << "vroom\n"; }
};

class Car {
    Engine engine_;  // Car HAS an engine
public:
    Car()  { std::cout << "Car ready\n"; }
    ~Car() { std::cout << "Car gone\n"; }
    void drive() { engine_.start(); }
}
```



A heirarchical structure in composite pattern

Inheritance vs Composition

- Inheritance heeft een "is-een" relatie
- Voorbeelden:
- Hond is een dier
- Telefoon is een apparaat
- Student is een mens

- Composition heeft een "heeft-een" relatie
- Voorbeelden:
  - Hond heeft een staart
  - Phone heeft een camera
  - Student heeft een OV-chipkaart

The Good, the Bad and the Ugly

Advantage

- **Simpliciteit**
- **Hiërarchieondersteuning**
- **Uniforme interface**
- **Uitbreidbaarheid**
- **Vereenvoudigde client-code**

The Good, the Bad and the Ugly

DisAdvantage

- **Toegenomen complexiteit & inefficiëntie**
- **Overhead**
- **Gemeenschappelijke interface**
- **Child element**
- **Gebrek aan hiërarchie**
- **Zoektijd**
- **Code duplicatie**

The Good, the Bad and the Ugly

Problems

- **Delen van child-objecten**
- **Extra abstractielagen**
- **Geheugen Kosten**
- **Code onoverzichtelijk met diepe nesting**
- **Recursie operaties zijn complexer**

Code voorbeelden

```
class CPU {
public:
    void process() {}
};

class RAM {
public:
    void load() {}
};

class Computer {
private:
    CPU cpu;
    RAM ram;

public:
    void boot() {
        ram.load();
        cpu.process();
    }
};
```

```
class Teacher {
public:
    void teach() {}
};

class Department {
private:
    Teacher* teacher;

public:
    Department(Teacher* t) : teacher(t) {}

    void conductClass() {
        if (teacher) {
            teacher->teach();
        }
    }
};
```

Mededemogelijkgemaaktdoor:

- Het internet